

Motion Detection Alarm System

Sensor-Based IoT Simulation — AT89C51 Microcontroller

CodeAlpha Internship — Internet of Things (IoT) Domain
Task 2: Sensor-Based Simulation

Submitted by: Pradyumn Nagesh Dhere
Branch: Electronics and Computer Science (Third Year)
Institution: Dbatu University

1. Objective

The objective of this task is to simulate a sensor-based IoT system in Proteus without physical hardware, as required under Task 2 of the CodeAlpha IoT internship. This project implements a **motion-triggered alarm system**: when motion is detected by the sensor input, an LED turns ON and a buzzer produces a repeated beep; when no motion is detected, both outputs remain OFF.

2. Components Used

Component	Function
AT89C51 (8051)	Main microcontroller running the control logic
LOGICSTATE (motion input)	Simulates a motion sensor / PIR output on pin P3.2
LED	Visual indicator, connected to P2.0
Buzzer	Audible alarm, connected to P2.1
Crystal 11.0592MHz + 2x33pF capacitors	Clock source on XTAL1 / XTAL2
10k resistor + capacitor	Power-on reset circuit on pin 9 (RST)

3. Circuit Diagram and Simulation Output (Proteus)

The circuit below shows the AT89C51 microcontroller with its crystal oscillator, reset circuit, motion input on P3.2, and LED/buzzer outputs on P2.0 and P2.1. Two simulation states are captured below: motion detected (state = 1) and no motion (state = 0).

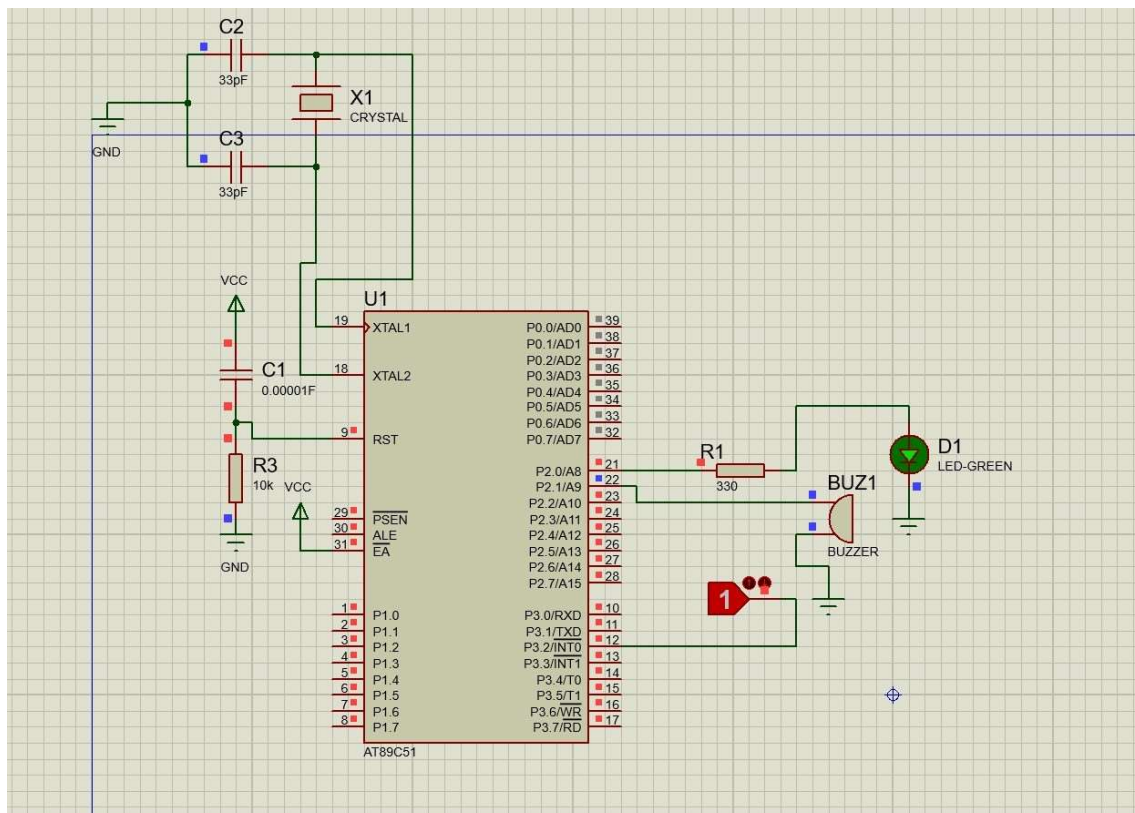


Figure 1: Simulation state — Motion detected (LOGICSTATE = 1). LED ON, buzzer active.

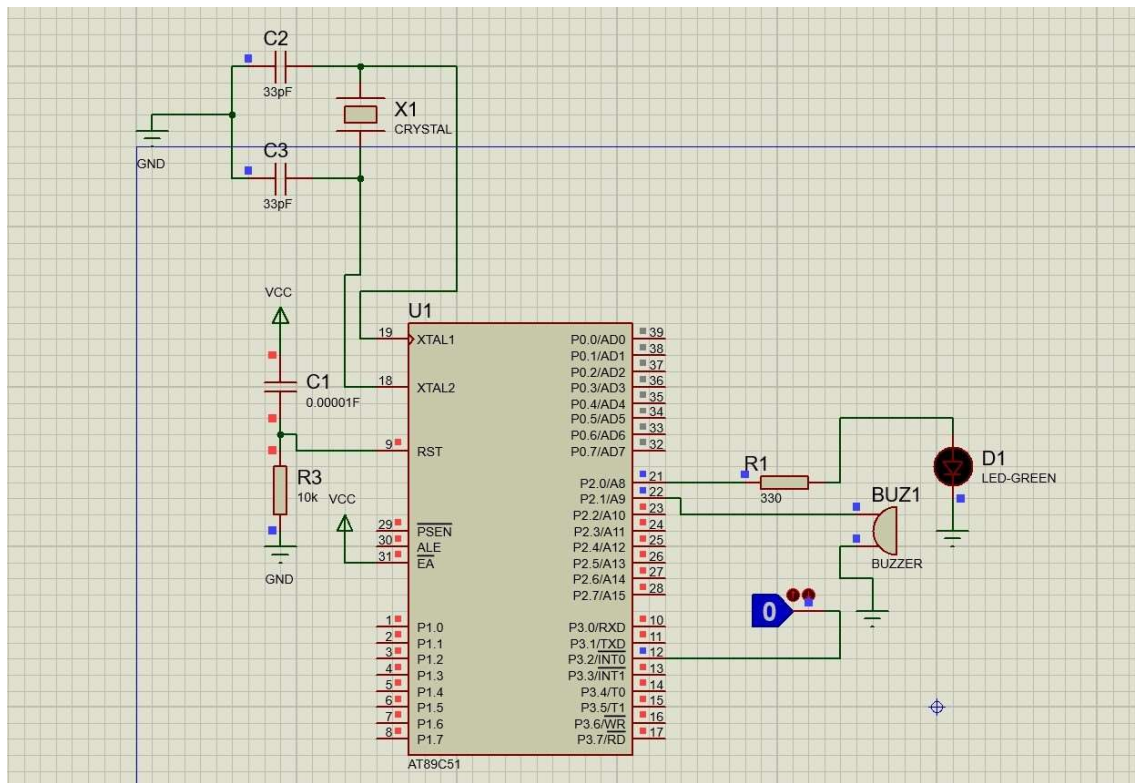


Figure 2: Simulation state — No motion ($LOGICSTATE = 0$). LED and buzzer OFF.

4. Source Code (Keil μ vision)

Project Title: Motion Detection Alarm System

```
#include <reg51.h> // Header file for 8051 microcontroller

// Pin definitions

sbit MOTION = P3^2; // Motion sensor input (LOGICSTATE)

sbit LED = P2^0;    // LED output

sbit BUZZER = P2^1; // Buzzer output

// Delay function // Generates an approximate software delay
void delay(unsigned int t)
{
    unsigned int i, j;
    for(i = 0; i < t; i++)
    {
        for(j = 0; j < 1275; j++);
    }
}

// Main program void
main(void) {
    // Configure input and initialize outputs
    MOTION = 1;    // Set P3.2 as input
    LED = 0;       // LED initially OFF

    BUZZER = 0;    // Buzzer initially OFF

    while(1) {     // Check whether motion is detected

        if(MOTION == 1) {
```

```
    LED = 1;    // Turn ON LED

    BUZZER = 1;    // Turn ON buzzer

    delay(200);    // Buzzer ON duration

    BUZZER = 0;    // Turn OFF buzzer

    delay(200);    // Pause before next beep

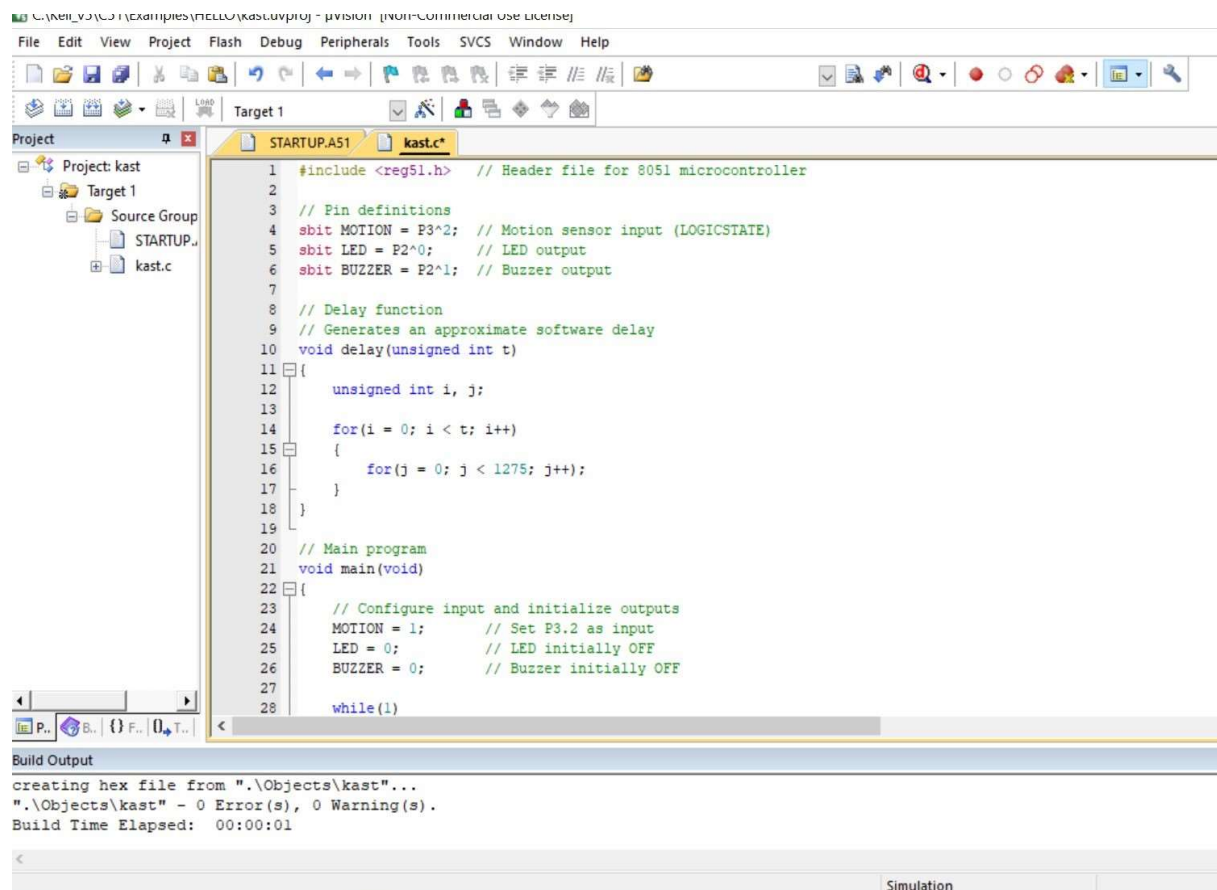
    }    else    {
    // No motion detected
    LED = 0;    // Turn OFF LED
    BUZZER = 0;    // Turn OFF buzzer
    }
}
}
```

5. Code Explanation

- **Pin setup:** P3.2 is used as the digital input from the motion sensor (LOGICSTATE), P2.0 drives the LED, and P2.1 drives the buzzer.
- **Initialization:** On startup, the LED and buzzer are set OFF and the motion pin is configured as input.
- **Main loop:** The program continuously polls the MOTION pin.
- **Motion detected (MOTION = 1):** The LED is switched ON and the buzzer is toggled ON and OFF with a delay, producing a repeated beep pattern.
- **No motion (MOTION = 0):** Both LED and buzzer are turned OFF.
- **Delay function:** A simple nested-loop software delay is used to create the timing gaps between buzzer beeps.

6. Keil Compilation Output

The code was written and compiled in Keil uVision (C51 compiler) targeting the AT89C51. The build completed with 0 errors and 0 warnings, and the generated .hex file was loaded into the AT89C51 in Proteus for simulation.



```
1 #include <reg51.h> // Header file for 8051 microcontroller
2
3 // Pin definitions
4 sbit MOTION = P3^2; // Motion sensor input (LOGICSTATE)
5 sbit LED = P2^0; // LED output
6 sbit BUZZER = P2^1; // Buzzer output
7
8 // Delay function
9 // Generates an approximate software delay
10 void delay(unsigned int t)
11 {
12     unsigned int i, j;
13
14     for(i = 0; i < t; i++)
15     {
16         for(j = 0; j < 1275; j++);
17     }
18 }
19
20 // Main program
21 void main(void)
22 {
23     // Configure input and initialize outputs
24     MOTION = 1; // Set P3.2 as input
25     LED = 0; // LED initially OFF
26     BUZZER = 0; // Buzzer initially OFF
27
28     while(1)
```

Build Output

```
creating hex file from ".\Objects\kast"...
".\Objects\kast" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:01
```

Simulation

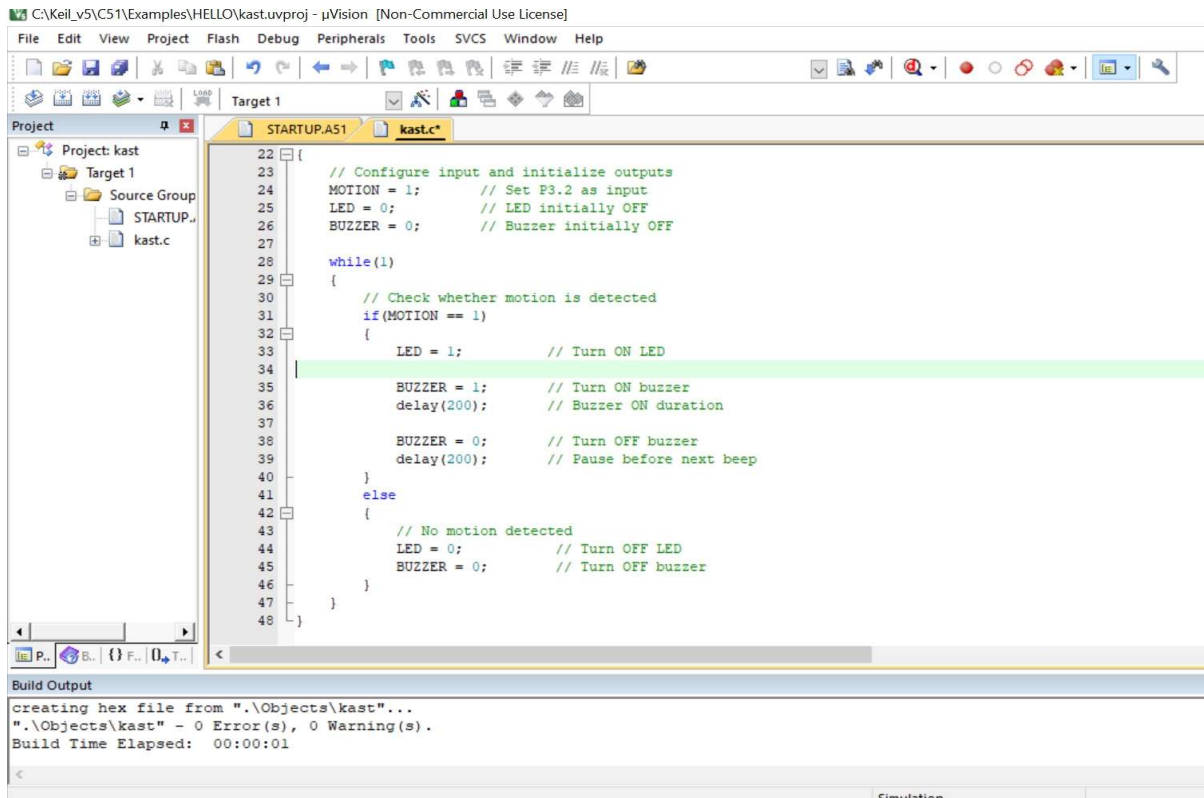


Figure 3: Keil uVision build output showing successful compilation (0 Errors, 0 Warnings).

7. Conclusion

This project successfully demonstrates a sensor-based IoT simulation using the AT89C51 microcontroller in Proteus, fulfilling the requirements of Task 2 of the CodeAlpha IoT internship. The system reads a digital motion input and automatically controls an LED and buzzer without any manual intervention, which is the core principle behind real-world motion-triggered alarms and automatic security lighting systems. Through this task, practical exposure was gained in embedded C programming, 8051 microcontroller I/O handling, circuit design in Proteus, and the use of Keil uVision for compiling and debugging embedded code.

8. Real-World Applications

- Home and office security systems (intruder detection).
- Automatic outdoor/staircase lighting triggered by movement.
- Parking lot and warehouse surveillance alerts.
- Smart doorbell / entry alert systems.